

FinWallet Ana Spesifikasyonu

1. Ürün amacı

FinWallet; .NET 8 tabanlı, multi-currency bir digital-wallet side projectidir. Amaç gerçek finansal backend problemlerini göstermektir: transaction consistency, double-entry accounting, idempotency, concurrency, external integrations, fraud, authentication/session security, cutoff, campaigns, notifications, reconciliation, security ve observability.

2. Güncel kapsam

Customer/Auth

- TR/AZ country/phone policy ile registration.
- Redis tabanlı OTP verification.
- PBKDF2 password hashing.
- JWT access token + durable server session.
- Opaque refresh-token rotation ve reuse detection.

Wallet/Account

- TRY/USD/EUR wallet create/list.
- FakeBank üzerinden external BankAccount opening.
- Wallet ve BankAccount ayrı domain kavramlarıdır.
- v1'de FX conversion yoktur.

Financial Core

- Double-entry Ledger.
- Durable FinancialTransaction.
- MSSQL tabanlı durable idempotency.
- Atomik wallet-to-wallet transfer.
- Internal + external fraud decision.

Platform

- YARP Gateway tüm normal public/client ve FinWallet->provider HTTP trafiğinin giriş noktasıdır.
- Gateway JWT, internal-service authorization, rate limit, health checks ve load balancing uygular.
- Tüm Web API projelerinde Swagger vardır; production'da varsayılan kapalıdır.

3. Henüz tamamlanmamış kapsam

- Public BankDeposit ve BankWithdrawal.
- Merchant purchase / campaign accounting.
- Public Refund/Reversal flow.
- Durable FraudEvents/manual review.
- Transactional Outbox/Inbox.
- Transaction history/read model.
- ReconciliationRuns/ReconciliationIssues.

- Merkezi maskeli structured logging, OpenTelemetry ve alerting.
- Real MSSQL/Redis/YARP integration-concurrency test suite.
- Logout/session-revoke public endpoint.

4. Teknoloji kuralları

- .NET 8 / C# 12.
- ASP.NET Core controller-based Web API.
- MSSQL durable source of truth.
- Redis yalnız transient state.
- JWT auth; ASP.NET Core Identity kullanılmaz.
- YARP reverse proxy/gateway.
- Built-in DI.
- Paid/freemium NuGet yasaktır.
- Paket versiyonları Directory.Packages.props ile merkezi yönetilir.

5. Mimari

Ana uygulama modular monolith'tir:

```
FinWallet.Api
FinWallet.Application
FinWallet.Domain
FinWallet.Infrastructure
FinWallet.Shared.Contracts
FinWallet.Shared.Web
FinWallet.Gateway
```

External simulatorlar ayrı Web API prosesleridir:

```
FakeBank.Api
FakeFraud.Api
FakeCutoff.Api
FakeCampaign.Api
FakeCommunication.Api
```

6. Finansal doğruluk invariant'ları

- 1 Hiçbir para hareketi Ledger'ı atlayamaz.
- 2 Her posted journal için total Debit = total Credit olmalıdır.
- 3 MSSQL final financial consistency authority'dir.
- 4 Redis kaybı duplicate money, negative balance veya ledger corruption yaratmamalıdır.
- 5 External HTTP açık financial SQL transaction içinde çalıştırılmaz.
- 6 Duplicate command ve retry güvenli olmalıdır.
- 7 Same idempotency key + different payload conflict'tir.
- 8 Completed financial history mutate/delete edilmez; correction reversal/compensation ile yapılır.
- 9 Currency her Money değerinin parçasıdır ve commit öncesi doğrulanır.
- 10 Reconciliation mismatch'i sessizce balance update ederek düzeltmez.

7. Gateway güvenlik kuralı

```
Client -> Gateway -> FinWallet.Api
FinWallet.Api -> Gateway /providers/* -> Fake providers
```

- Protected public /api/* rotalarında Gateway JWT ister.
- FinWallet.Api JWT/ownership kontrolünü tekrar yapar.
- FinWallet.Api provider route'larına InternalServiceKey ile gider.
- Gateway destination request'e ayrı DownstreamServiceKey yazar.
- Backend/provider business endpointleri downstream key olmadan doğrudan çağrıyı reddeder.

8. Finansal işlem modeli

Wallet transfer sırası:

```
completed durable replay
-> durable session/risk signals
-> internal fraud
-> external fraud
-> final Allow
-> atomic MSSQL posting
```

Atomic posting aynı transaction'da idempotency + balances + FinancialTransaction + LedgerJournal + LedgerEntries commit eder.

9. Security kuralları

- Password/OTP/JWT/refresh token/service key/connection secret loglanmaz.
- SQL parameterized kullanılır.
- Auth/ownership client-supplied ID veya risk flag'e güvenmez.
- JWT signing algorithm code-level invariant'tır.
- PBKDF2 scheme değişikliği versioned migration gerektirir.
- Rate/body/header/connection/timeouts Gateway ve backend katmanlarında uygulanır.
- Volumetric DDoS için ayrıca ingress/WAF/cloud DDoS control gerekir.

10. Definition of Done

Bir feature ancak şu durumda tamamlanır:

- solution build başarılı;
- ilgili testler başarılı;
- financial/concurrency/idempotency invariant'ları korunmuş;
- external failure davranışı tanımlanmış;
- güvenlik/logging etkileri değerlendirilmiş;
- package inventory güncel;
- TR/EN XML documentation güncel;
- etkilenen tüm Markdown dokümanları TR+EN ve kodla tutarlı.

11. v1 kapsam dışı

- gerçek banka/fraud/SMS/email providerları;
- credit-card acquiring/payment gateway;
- full core banking;
- loans/credit scoring;
- stock/crypto trading;
- FX conversion;

- Event Sourcing;
- zorunlu Kafka/RabbitMQ;
- full microservice decomposition.